

Московский государственный технический университет

Факультет ИУ «Информатика и системы управления»

Кафедра ИУ-1 «Системы автоматического управления»

ОТЧЕТ

по лабораторной работе №2

по дисциплине

«Микропроцессорные и встраиваемые системы»

Выполнил: Салженикин Д.А.

Галиев Э.Р.

Группа: ИУ1-52Б

Проверила: Сычев В.В.

Работа выполнена:

Отчет сдан:

Оценка:

Цель работы: Приобретение практических навыков программирования на Си и отработка в среде разработки STM32CubeIDE, разработать и запустить программу для микроконтроллера STM32L476

Задачи: Настройка тактирования и прерываний, настройка пинов GPIO, инициализация SPI, настройка USART, прием символов через USART и отправка их обратно.

Код:

```
#include "main.h"
#include "stm32l476xx.h"

extern uint32_t systemTimeCounter;

void delayMs(uint32_t delay)
{
   uint32_t curr = systemTimeCounter;
   while(systemTimeCounter - curr < delay)
   {   }
}


int main(void)
{
   // PWR AND SYS ENABLE
   RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
   RCC->APB1ENR1 |= RCC_APB1ENR1_PWREN;
   RCC->CR |= RCC_CR_MSIRGSEL;
   SysTick_Config(4000000/1000);
   NVIC_SetPriority(SysTick_IRQn,0);
   NVIC_EnableIRQ(SysTick_IRQn);
   // GPIO_SW
   RCC->AHB2ENR |= RCC_AHB2ENR_GPIOEEN;
   GPIOE->MODER &= ~((3 << GPIO_MODER_MODE11_Pos)| (3
<<GPIO_MODER_MODE12_Pos ) | (3 <<GPIO_MODER_MODE10_Pos));
   GPIOE->OSPEEDR |= ((GPIO_OSPEEDER_OSPEEDR10) |
(GPIO_OSPEEDER_OSPEEDR11) | (GPIO_OSPEEDER_OSPEEDR12));
   //  GPIOE->IDR |= GPIO_IDR_ID15;

   //LED
```

```
   GPIOE->MODER &= ~((3 << GPIO_MODER_MODE13_Pos)| (3
<<GPIO_MODER_MODE14_Pos ) | (3 <<GPIO_MODER_MODE15_Pos));
   GPIOE->MODER |= ((1 << GPIO_MODER_MODE13_Pos)| (1
<<GPIO_MODER_MODE14_Pos ) | (1 <<GPIO_MODER_MODE15_Pos));
   GPIOE->OSPEEDR |= ((GPIO_OSPEEDER_OSPEEDR13) |
(GPIO_OSPEEDER_OSPEEDR14) | (GPIO_OSPEEDER_OSPEEDR15));

   // USART
   RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;
   GPIOB->MODER &= ~((3 << GPIO_MODER_MODE6_Pos)| (3
<<GPIO_MODER_MODE7_Pos ));
   GPIOB->MODER |= (2 << GPIO_MODER_MODE6_Pos)| (2
<<GPIO_MODER_MODE7_Pos );
   GPIOB->OSPEEDR |=  ((GPIO_OSPEEDER_OSPEEDR6) |
(GPIO_OSPEEDER_OSPEEDR7));
   GPIOB->AFR[0] |= (7 << GPIO_AFRL_AFSEL6_Pos) | (7 <<
GPIO_AFRL_AFSEL7_Pos);

   RCC->APB2ENR |= RCC_APB2ENR_USART1EN;
   RCC->CCIPR &= ~(RCC_CCIPR_USART1SEL);


   USART1->CR1 &= ~(USART_CR1_M0 | USART_CR1_M1);
   USART1->CR1 |= USART_CR1_OVER8;
   USART1->BRR |= 0x42;
   USART1->CR2 &= ~(USART_CR2_STOP);
   USART1->CR1 |= USART_CR1_UE;
   USART1->CR1 |= USART_CR1_TE | USART_CR1_RE;

   char str[] = "abcde";
   for(int i =0; str[i];i++)
   {
      while(!(USART1->ISR & USART_ISR_TXE));
      USART1->TDR = str[i];
   }


  // Led

   GPIOE->ODR |= GPIO_ODR_OD13;
   GPIOE->ODR |= GPIO_ODR_OD14;
   GPIOE->ODR |= GPIO_ODR_OD15;
```

```
uint8_t mode = 0;

  while (1)
  {
     if(!(GPIOE->IDR & GPIO_IDR_ID13))
     {
        mode = 1;
        GPIOE->ODR &= ~GPIO_ODR_OD10;
        GPIOE->ODR |= GPIO_ODR_OD11;
        GPIOE->ODR |= GPIO_ODR_OD12;

     }
     if(!(GPIOE->IDR & GPIO_IDR_ID14))
     {
        mode = 2;
        GPIOE->ODR |= GPIO_ODR_OD10;
        GPIOE->ODR &= ~GPIO_ODR_OD11;
        GPIOE->ODR |= GPIO_ODR_OD12;

     }
     if(!(GPIOE->IDR & GPIO_IDR_ID15))
     {
        mode = 3;
        GPIOE->ODR |= GPIO_ODR_OD10;
        GPIOE->ODR |= GPIO_ODR_OD11;
        GPIOE->ODR &= ~GPIO_ODR_OD12;

     }
     delayMs(1000);
     if (mode == 1){
       if (USART1->ISR & USART_ISR_RXNE) {
          uint32_t symbol = USART1->RDR;
          while(!(USART1->ISR & USART_ISR_TXE));
          USART1->TDR = symbol;
       }
     }
     if (mode == 2){
       if (USART1->ISR & USART_ISR_RXNE) {
          uint32_t symbol = USART1->RDR;
          while(!(USART1->ISR & USART_ISR_TXE));
          USART1->TDR = symbol;
          USART1->TDR = symbol;
          USART1->TDR = symbol;
```

```
    }
   }
   if (mode == 3){
    if (USART1->ISR & USART_ISR_RXNE) {
       uint32_t symbol = USART1->RDR;
       while(!(USART1->ISR & USART_ISR_TXE));
       USART1->TDR = symbol;
       USART1->TDR = symbol;
       USART1->TDR = symbol;
       USART1->TDR = symbol;
       USART1->TDR = symbol;
    }
   }

 }

}
```

Пояснения к коду:

1. Настройка тактирования и прерываний:
   - Включение тактирования для модуля системных настроек и управления энергопотреблением (SYSCFG).
   - Включение тактирования для модуля питания (PWR).
   - Установка источника тактирования в режиме MSI (Multiple Speed Internal oscillator) с использованием регистра RCC_CR.
   - Настройка системного таймера SysTick с частотой 4 МГц.
   - Установка приоритета прерывания SysTick.
   - Разрешение прерывания SysTick.
2. Настройка пинов GPIO:
   - Включение тактирования для порта GPIOE.
   - Настройка ряда пинов GPIOE для различных режимов работы (вывод общего назначения, скорость работы).
   - Настройка пинов для светодиодов и кнопок.
3. Настройка USART (универсального средства асинхронной передачи данных):
   - Включение тактирования для порта GPIOB.
   - Настройка пинов GPIOB в альтернативном режиме для использования USART.
   - Установка параметров USART, таких как скорость передачи и биты данных.
   - Разрешение работы USART для передачи и приема данных.
   - Отправка строки "abcde" через USART.

4. Основной цикл:
    - В бесконечном цикле выполняется следующее:
        - Ожидание нажатия кнопок и установка переменной mode в соответствии с нажатой кнопкой.
        - Изменение состояния светодиодов в зависимости от mode.
        - Задержка в 1000 миллисекунд с использованием функции delayMs.
        - В зависимости от mode, прием символов через USART и отправка их обратно с разным количеством повторений.

Вывод:

В ходе данной работы была разработана программа для микроконтроллера STM32L476, предназначенная для настройки различных периферийных устройств, включая системный таймер, порты GPIO, USART (универсальное средство асинхронной передачи данных), а также светодиоды и кнопки.